

Encapsulamento e o jogo

O objeto passa a proteger o próprio estado, o jogador escolhe o personagem, o chefe final aparece, e o RPG vira um jogo do começo ao fim.

Continuação direta da parte 1.

Você precisa das classes Guerreiro, Mago, Goblin, Orc e Troll funcionando antes de começar.

Onde você está

Na parte 1 você aprendeu herança e polimorfismo. Saiu de uma classe só para uma família de classes, e o laço de combate passou a funcionar com qualquer uma delas sem saber qual é.

Confira que você tem isto antes de seguir:

ARQUIVO	O QUE PRECISA TER
<code>personagem.py</code>	A classe <code>Personagem</code> , ainda sem alterações.
<code>classes.py</code>	<code>Guerreiro</code> , <code>Mago</code> , <code>Goblin</code> , <code>Orc</code> e <code>Troll</code> .
<code>main.py</code>	O combate rodando com <code>[Goblin(), Orc(), Troll()]</code> .

ATENÇÃO, AGORA É DIFERENTE

Na parte 1 o `personagem.py` ficava intocado. **A partir daqui você vai editá-lo.** Antes de começar, faça uma cópia de segurança do arquivo, com o nome `personagem_backup.py`. Se algo der muito errado, você volta.

O caminho

1 Encapsulamento

Por que `inimigo.vida = -999` não deveria funcionar, e como fechar essa porta.

2 O estado que não expira

Um bug de verdade no seu jogo, achado com o que você aprendeu agora.

3 Seleção de personagem

O jogador escolhe entre Guerreiro e Mago. Polimorfismo virando menu.

4 O chefe final

Um Dragão com um golpe especial a cada três turnos.

5 Fechar o jogo

Descanso entre lutas, tela final, e a entrega.

Lembrete: os extras do anexo B continuam opcionais. Nenhum capítulo depende deles.

Sumário

01	Encapsulamento	4
02	O estado que não expira	8
03	Seleção de personagem	11
04	O chefe final	13
05	Fechar o jogo	15
AN	Anexo A: erros que você vai ver	17
AN	Anexo B: extras	18
AN	Anexo C: checklist de entrega	20

CAPÍTULO 1

Encapsulamento

Encapsulamento tem uma regra só: **cada objeto cuida do próprio estado**. Quem está de fora não altera os atributos direto. Pede a mudança por um método.

Você já faz isso sem saber o nome. Olhe o `receber_dano`:

```
def receber_dano(self, quantidade):
    dano_real = quantidade - self.defesa

    if self.defendendo:
        dano_real = dano_real // 2

    dano_real = max(0, dano_real)
    self.vida = max(0, self.vida - dano_real)
```

Esse método garante quatro regras: desconta a defesa, corta o dano pela metade se estiver defendendo, nunca deixa o dano ficar negativo, e nunca deixa a vida ficar abaixo de zero.

Quem chama `inimigo.receber_dano(30)` não precisa saber de nada disso. O objeto aplica as regras sozinho.

O atalho que estraga tudo

O problema é que Python deixa fazer isto:

```
inimigo.vida = inimigo.vida - 30 # pula todas as regras acima
inimigo.vida = -999              # estado impossível
```

Teste no seu jogo. Crie um Goblin e escreva `goblin.vida = -999`, depois `goblin.mostrar_status()`:

SAÍDA

```
Goblin
Vida: -999 / 40
Poções: 0
```

Um objeto com vida negativa. A defesa foi ignorada, o `max(0, ...)` foi ignorado. O método existe para ninguém, porque tem uma porta aberta do lado.

Fechando a porta

Python não tem atributo privado de verdade. O que existe é uma **convenção**: um atributo que começa com underscore, como `_vida`, significa "isto é assunto interno da classe, não mexa de fora".

Ninguém é impedido pela linguagem. Mas quem escreve `inimigo._vida = -999` está fazendo isso de propósito, e sabe que está.

PASSO 1: RENOMEAR NO `PERSONAGEM.PY`

Troque `self.vida` por `self._vida` e `self.vida_maxima` por `self._vida_maxima`, **em todo o arquivo**. No VS Code, selecione o texto e use Ctrl+H para substituir.

Os lugares afetados são o `__init__`, o `mostrar_status`, o `esta_vivo`, o `receber_dano` e o `usar_pocao`.

PASSO 2: CRIAR A PORTA OFICIAL

Se ninguém pode mexer na vida de fora, precisa existir um método para curar. Acrescente na classe `Personagem`:

```
def curar(self, quantidade):
    if quantidade <= 0:
        return
    self._vida = min(self._vida + quantidade, self._vida_maxima)
```

Ele protege duas regras: não aceita cura negativa (que seria dano disfarçado) e não passa da vida máxima.

PASSO 3: USAR A PORTA NO PRÓPRIO PERSONAGEM

Dentro do `usar_pocao`, troque a linha que mexia na vida direto:

```
self._vida = min(self._vida + 25, self._vida_maxima)
```

```
self.curar(25)
```

PASSO 4: CONSERTAR O MAGO

Agora o seu `classes.py` está quebrado, e é por um bom motivo. O Mago mexia na vida direto:

```
def defender(self):
    super().defender()
    self.vida = min(self.vida + 10, self.vida_maxima)
    print(self.nome, "recuperou 10 de vida")
```

```
def defender(self):
    super().defender()
    self.curar(10)
    print(self.nome, "recuperou 10 de vida")
```

Repare no que aconteceu: o Mago **diminuiu**. Ele não precisa mais saber que existe vida máxima, nem lembrar de usar `min`. Essa regra é assunto do Personagem. O Mago só diz quanto quer curar.

O SINAL DE QUE DEU CERTO

Encapsulamento bem feito faz as outras classes ficarem *menores*, não maiores. Se depois de encapsular o seu código cresceu em todo lugar, provavelmente você fechou a porta sem construir a porta nova.

O `main.py` não precisou de nada

Abra o `main.py` e procure por `.vida`. Não tem.

Ele só usa `esta_vivo()`, `mostrar_status()`, `atacar()`, `defender()` e `usar_pocao()`. Ele já pedia tudo por método. Por isso a mudança de hoje não quebrou nada ali.

Isso não foi sorte. É o que acontece quando o código de fora respeita o objeto: você pode mudar o interior da classe inteira sem quebrar quem usa.

CHECKPOINT 1

No `personagem.py` não existe mais nenhum `self.vida`, só `self._vida`.

Existe o método `curar`, e o `usar_pocao` usa ele.

O Mago usa `self.curar(10)`.

O jogo roda normalmente.

TESTE DO CHECKPOINT 1

```
g = Goblin()
print(g.vida)
```

Resultado esperado:

```
AttributeError: 'Goblin' object has no attribute 'vida'
```

O erro é o sucesso. A porta velha não existe mais.

Agora teste a porta nova:

```
m = Mago("Elric")
m.curar(-500)
m.mostrar_status()
```

Resultado esperado: a vida continua 70. O método recusou a cura negativa.

Se a vida diminuiu: falta o `if quantidade <= 0: return` no seu `curar`.

LEIA A FONTE

Documentação oficial em português, seção **9.6 Variáveis privadas**:

docs.python.org/pt-br/3/tutorial/classes.html

A documentação é bem direta sobre o que o underscore realmente faz. Segundo ela, existe atributo verdadeiramente privado em Python? Responda com as palavras dela:

A seção também cita um segundo uso do underscore, com dois underscores no início. Como se chama o que o Python faz nesse caso?

Terminou o checkpoint 1? Extras: anexo B, itens **E1.1** e **E1.2**.

CAPÍTULO 2

O estado que não expira

Agora que você sabe olhar para estado de objeto, dá para achar um bug que está no seu jogo desde o Dia 5.

Ache o bug

Olhe o `receber_dano`. O `self.defendendo` volta para `False` ali dentro. E o `atacar`:

```
def atacar(self, alvo):
    rolagem = random.randint(1, 20)

    if rolagem < 5:
        print(self.nome, "errou o ataque")
        return # sai sem chamar receber_dano
```

Pergunta: se o inimigo **erra** o ataque, quem desliga o `defendendo` do jogador?

Ninguém. O `receber_dano` não foi chamado.

Confira você mesmo:

```
h = Guerreiro("Thoric")
h.defender()
print("defendendo:", h.defendendo)

# o inimigo erra o ataque, entao receber_dano nunca roda
print("depois do inimigo errar:", h.defendendo)
```

SAÍDA

```
Thoric preparou a defesa
defendendo: True
depois do inimigo errar: True
```

Na prática, "defender" no seu jogo não quer dizer "me protejo neste turno". Quer dizer "fico protegido até alguém me acertar", o que pode durar cinco turnos. É um estado inválido que sobrevive.

Por que a correção não é onde você imagina

A tentação é consertar no `main.py`:

```
jogador.defendendo = False      # de fora, mexendo no atributo
turno_do_jogador(jogador, inimigo)
```

Isso funciona e está errado. É exatamente o atalho do capítulo 1: código de fora mexendo no estado interno do objeto. Se amanhã defender passar a gastar energia, o `main.py` vai ter que saber disso também.

Quem tem que zerar o estado do personagem é o personagem. Acrescente na classe `Personagem`:

```
def iniciar_turno(self):
    self.defendendo = False
```

E tire a linha do `receber_dano`, que agora não é mais responsabilidade dele:

```
if self.defendendo:
    dano_real = dano_real // 2
    self.defendendo = False      # APAGUE esta linha
```

No `main.py`, dentro do `combate`, cada um avisa o próprio começo de turno:

```
jogador.iniciar_turno()
turno_do_jogador(jogador, inimigo)

if inimigo.esta_vivo():
    inimigo.iniciar_turno()
    inimigo.atacar(jogador)
```

CHECKPOINT 2

- Existe o método `iniciar_turno` no `Personagem`.
- O `receber_dano` não mexe mais no `defendendo`.
- O `combate` chama `iniciar_turno()` nos dois lados.

TESTE DO CHECKPOINT 2

```
h = Guerreiro("Thoric")
h.defender()
print("depois de defender:", h.defendendo)
h.iniciar_turno()
print("depois de iniciar_turno:", h.defendendo)
```

Resultado esperado:

```
Thoric preparou a defesa
depois de defender: True
depois de iniciar_turno: False
```

Se o segundo continuar `True` : o `iniciar_turno` não está sendo chamado, ou ficou escrito fora da classe. Confira a indentação.

Terminou o checkpoint 2? Extras: anexo B, item **E2.1**.

CAPÍTULO 3

Seleção de personagem

Até agora o jogador sempre foi um Guerreiro fixo no código. Vamos deixar ele escolher.

Este capítulo é a recompensa da parte 1: escolher um personagem **é** instanciar uma sub-classe diferente.

A ideia nova: guardar a classe

Você já guardou números, textos e objetos em variáveis. Uma classe também pode ser guardada:

```
escolhida = Guerreiro          # sem parenteses: a CLASSE, nao um objeto
jogador = escolhida("Thoric") # agora sim, cria o objeto
```

Repare na diferença. `Guerreiro` é o molde. `Guerreiro("Thoric")` é uma pessoa feita com o molde.

Com isso dá para montar um dicionário de opções, no topo do `main.py`:

```
CLASSES = {
    "1": Guerreiro,
    "2": Mago,
}
```

O menu

```
def escolher_personagem():
    print("Escolha o seu personagem:")
    for tecla, classe in CLASSES.items():
        print(tecla, "-", classe.__name__)

    escolha = input("Opção: ")
    while escolha not in CLASSES:
        print("Opção inválida")
        escolha = input("Opção: ")

    nome = input("Nome: ")
    return CLASSES[escolha](nome)
```

Duas coisas para reparar:

`classe.__name__` devolve o nome da classe como texto, então o menu se escreve sozinho. Você não digita "Guerreiro" em lugar nenhum.

E o `while escolha not in CLASSES` repete até a opção existir. O jogo não começa com escolha inválida.

No `main`, troque a linha que criava o jogador:

```
jogador = Personagem("Thoric", 100, 15, 5, 2)
```

```
jogador = escolher_personagem()
```

CHECKPOINT 3

O jogo começa perguntando qual personagem e qual nome.

O menu é montado a partir do dicionário `CLASSES`, não digitado à mão.

Opção inválida faz perguntar de novo, sem começar o jogo.

TESTE DO CHECKPOINT 3

Acrescente **uma linha só** no dicionário:

```
CLASSES = {
    "1": Guerreiro,
    "2": Mago,
    "3": Troll,      # so para testar
}
```

Resultado esperado: o menu passa a mostrar três opções, e escolher a 3 deixa você jogar como Troll.

A pergunta do teste: quantas linhas *fora do dicionário* você precisou mudar?

A resposta certa é **nenhuma**. Se você precisou escrever um `print` novo ou um `if`, o seu menu não está sendo gerado pelo dicionário.

Depois do teste, tire o Troll da lista.

Terminou o checkpoint 3? Extras: anexo B, itens **E3.1** e **E3.2**.

CAPÍTULO 4

O chefe final

Um chefe precisa de alguma coisa que os outros inimigos não têm. O Dragão tem um golpe que sai a cada três ataques dele, e esse golpe ignora a defesa.

Para contar os turnos, ele precisa guardar um número entre um ataque e outro. Esse número é assunto interno dele: começa com underscore.

```
class Dragao(Personagem):
    def __init__(self):
        super().__init__("Dragão Ancião", 110, 14, 4, 0)
        self._turnos = 0

    def atacar(self, alvo):
        self._turnos += 1

        if self._turnos % 3 == 0:
            dano = self.ataque + 8
            print("O Dragão solta um sopro de fogo!")
            alvo.receber_dano(dano + alvo.defesa)
            print("O sopro causou", dano, "de dano e ignorou a defesa")
            return

        super().atacar(alvo)
```

Três coisas acontecem aqui, e todas são da parte 1 ou do capítulo 1 deste PDF:

- O `self._turnos = 0` vem **depois** do `super().__init__(...)`. Atributo novo entra depois que a classe-mãe montou o objeto.
- Nos turnos normais, ele chama `super().atacar(alvo)`. Não reescreve o ataque comum.
- O truque de ignorar a defesa é somar a defesa do alvo antes de mandar. O `receber_dano` vai descontar, e sobra o valor cheio. As regras do alvo continuam valendo: o objeto não foi violado.

Coloque ele no fim da fila, no `main.py`:

```
inimigos = [Goblin(), Orc(), Troll(), Dragao()]
```

CHECKPOINT 4

O Dragão é o último inimigo da lista.

A cada três ataques dele sai o sopro de fogo.

Nos outros turnos ele ataca chamando `super().atacar(alvo)`.

TESTE DO CHECKPOINT 4

```
d = Dragao()
alvo = Guerreiro("Thoric")
for i in range(1, 7):
    print("-- ataque", i)
    d.atacar(alvo)
```

Resultado esperado: o sopro de fogo aparece no ataque 3 e no ataque 6, e não nos outros.

Se o sopro sair toda vez: confira o `% 3 == 0`.

Se der `AttributeError: '_turnos'`: você criou o `self._turnos` antes do `super().__init__(...)`, ou esqueceu dele.

Terminou o checkpoint 4? Extras: anexo B, item **E4.1**.

Fechar o jogo

Falta pouco. O jogo já tem escolha de personagem, quatro inimigos e um chefe. Falta o jogador sobreviver até lá e o jogo terminar direito.

Descanso entre as lutas

Sem nenhuma recuperação, ninguém chega ao Dragão. Depois de cada vitória, no `main`:

```
for inimigo in inimigos:
    venceu = combate(jogador, inimigo)
    if not venceu:
        print("Fim de jogo.")
        return
    jogador.curar(50)
    print(jogador.nome, "descansa e recupera 50 de vida")

print("\nVocê derrotou todos os inimigos. Fim.")
```

Repare que a cura passa pelo método `curar`, criado no capítulo 1. Em nenhum momento o `main.py` toca em `_vida`.

Está balanceado?

Com estes números, um Guerreiro que ataca, defende quando está abaixo de 40% e toma poção abaixo de 30% vence o jogo em pouco mais da metade das partidas. É difícil e é possível.

Se ficar fácil ou impossível demais para o seu gosto, mexa nos números. Balancear jogo é tentativa e erro, e agora todos os números estão em um lugar só: o `__init__` de cada classe.

CHECKPOINT 5, O ÚLTIMO

O jogo roda do início ao fim: escolhe personagem, enfrenta quatro inimigos em ordem, e termina com mensagem de vitória ou derrota.

Perder no meio encerra o jogo, sem travar e sem erro.

Nenhum arquivo acessa `_vida` de fora da classe `Personagem`.

TESTE FINAL

Abra o `main.py` e o `classes.py` e procure por `_vida` nos dois, com Ctrl+F.

Resultado esperado: nenhuma ocorrência nos dois arquivos.

Se aparecer alguma, tem código de fora mexendo no estado interno do personagem. Troque por uma chamada de método: `curar()` ou `receber_dano()`.

O que você construiu

CONCEITO	ONDE ESTÁ NO SEU JOGO
Classe e objeto	<code>Personagem</code> e cada inimigo criado a partir dela.
Herança	<code>Guerreiro</code> , <code>Mago</code> , <code>Goblin</code> , <code>Orc</code> , <code>Troll</code> e <code>Dragao</code> .
<code>super()</code>	Todos os <code>__init__</code> das filhas, o <code>defender</code> do Mago e o <code>atacar</code> do Dragão.
Polimorfismo	O <code>combate</code> chama <code>inimigo.atacar()</code> sem saber quem é.
Encapsulamento	<code>_vida</code> , e as portas <code>curar</code> e <code>receber_dano</code> .

São os quatro conceitos da matéria, todos rodando ao mesmo tempo, num programa que você joga.

Com o jogo fechado, os extras **E4.2** (inventário) e **E4.3** (salvar o jogo) ficam liberados. São os dois maiores do material.

ANEXO A

Erros que você vai ver

1. OBJECT HAS NO ATTRIBUTE 'VIDA'

```
AttributeError: 'Goblin' object has no attribute 'vida'
```

Depois do capítulo 1, isso pode ser *certo* ou *errado*.

É **certo** se você escreveu esse código de propósito para testar. A porta velha fechou.

É **erro** se aconteceu enquanto jogava. Nesse caso sobrou um `self.vida` em algum lugar que você não renomeou. Procure por `.vida` em todos os arquivos e veja qual ficou sem underscore.

2. RENOMEOU DEMAIS

```
AttributeError: 'Guerreiro' object has no attribute '_vida_maxima_maxima'
```

O Ctrl+H trocou `vida` dentro de `vida_maxima` também, gerando nomes duplicados. Desfaça com Ctrl+Z e substitua com mais cuidado: primeiro `self.vida_maxima` por `self._vida_maxima`, e só depois `self.vida` por `self._vida`.

3. O SOPRO DO DRAGÃO SAI TODA VEZ

Sem erro na tela, mas o chefe usa o especial em todos os turnos. Duas causas comuns:

- O `self._turnos += 1` ficou dentro do `if`, em vez de antes dele.
- Você escreveu `if self._turnos % 3` sem o `== 0`. Assim o teste dá verdadeiro sempre que o resto *não* for zero, que é o contrário do que você quer.

4. O JOGO TRAVA DEPOIS DA VITÓRIA

Se o jogo continua pedindo ação depois que o inimigo morreu, confira o `while` do `combate`. Ele precisa checar os dois lados:

```
while jogador.esta_vivo() and inimigo.esta_vivo():
```

Extras

Opcionais, como sempre. Os últimos três mexem em partes que o conteúdo principal já terminou de usar, então agora eles podem ser mais invasivos.

E1.1 AS POÇÕES TAMBÉM SÃO ESTADO

Depende de: *checkpoint 1*.

O atributo `pocoos` tem uma regra: nunca pode ficar negativo. Mas está aberto para qualquer um mexer.

Renomeie para `_pocoos` e crie um método `ganhar_pocao(self, quantidade)`. Ajuste o `usar_pocao` e o `mostrar_status`.

E1.2 O MAGO É IMORTAL

Depende de: *checkpoint 1*.

Descubra o exploit sozinho: pegue um Mago e mande ele defender repetidamente contra o Goblin, sem nunca atacar. Veja o que acontece com a vida dele.

Depois responda, com contas:

1. Quanto dano o Mago leva por turno defendendo contra um inimigo de ataque `A`? Escreva a fórmula usando o `receber_dano`.
2. Quanto ele cura por turno?
3. A partir de qual valor de ataque o inimigo consegue vencer essa disputa?
4. Algum inimigo do jogo chega nesse valor?

Agora proponha uma correção. Existem várias, e nenhuma é obviamente a certa: curar menos, curar só uma vez por luta, gastar poção para curar, ou dar mais ataque aos inimigos. Escolha uma, implemente e explique por que escolheu essa.

E2.1 DEFESA QUE DURA DOIS TURNOS

Depende de: *checkpoint 2*.

Crie uma subclasse cuja defesa dura dois turnos em vez de um. Dica: ela vai precisar de um contador próprio e vai sobrescrever `iniciar_turno`.

E3.1 MAIS UMA CLASSE JOGÁVEL

Depende de: checkpoint 3.

Crie uma classe jogável nova, com um comportamento próprio, e coloque no dicionário `CLASSES`. Confirme que só o dicionário precisou mudar.

Se você fez o extra E2.1 **do PDF da parte 1**, o seu Arqueiro já serve. Cuidado para não confundir com o E2.1 deste anexo.

E3.2 DIFICULDADE

Depende de: checkpoint 3.

Faça um segundo menu, no início, perguntando a dificuldade. No fácil, o jogador recupera 70 entre lutas. No difícil, 20.

Cuidado: a dificuldade não pode virar um `if` dentro do combate. Ela deve ser decidida uma vez e guardada.

E4.1 VENENO

Depende de: checkpoint 4 e do extra E2.1.

Crie um inimigo que envenena. Quem está envenenado perde 5 de vida no começo de cada turno, por três turnos.

Pense onde isso mora: o veneno é estado de quem foi envenenado, e o `iniciar_turno` já existe para acontecer no começo do turno.

E4.2 INVENTÁRIO

Depende de: checkpoint 5, jogo fechado.

Crie uma classe `Item` com nome e efeito, e dê ao Personagem uma lista de itens.

Antes de escrever, responda: `Item` deve herdar de `Personagem`? Use a regra do "é um".

E4.3 SALVAR O JOGO

Depende de: checkpoint 5, jogo fechado.

Ao fim de cada luta, grave o nome, a classe e a vida do jogador em um arquivo de texto. Ao abrir o jogo, ofereça continuar de onde parou.

Você vai precisar reconstruir o objeto a partir do texto. O dicionário `CLASSES` vai ajudar: ele já traduz um texto em uma classe.

ANEXO C

Checklist de entrega

Confira antes de entregar o projeto.

FUNCIONA

- ☐ O jogo roda do início ao fim sem erro na tela.
 - ☐ Dá para escolher o personagem e digitar o nome.
 - ☐ Opção inválida no menu não quebra nem começa o jogo.
 - ☐ Perder no meio encerra com mensagem, sem travar.
 - ☐ Ganhar do último inimigo mostra a mensagem final.
-

ORIENTAÇÃO A OBJETOS

- ☐ Existem pelo menos quatro classes que herdam de `Personagem`.
 - ☐ Todas chamam `super().__init__()` na primeira linha do `__init__`.
 - ☐ Pelo menos uma classe substitui um método, sem `super()`.
 - ☐ Pelo menos uma classe estende um método, chamando `super()`.
 - ☐ O `combate` não tem nenhum `if` perguntando o tipo do personagem.
 - ☐ Nenhuma hierarquia tem mais de um nível.
-

ENCAPSULAMENTO

- ☐ A vida é `_vida` dentro do `Personagem`.
 - ☐ `main.py` e `classes.py` não têm nenhum `_vida`.
 - ☐ Toda cura passa pelo método `curar`.
 - ☐ O `curar` recusa valor negativo e não passa da vida máxima.
-

ORGANIZAÇÃO

- ☐ Os três arquivos separados: `personagem.py`, `classes.py`, `main.py`.
 - ☐ Nenhum arquivo com nome contendo espaço ou acento.
 - ☐ O código roda com `python3 main.py`, sem precisar editar nada antes.
-

O ÚLTIMO TESTE

Peça para alguém que não programa jogar o seu jogo, sem você explicar nada. Se a pessoa souber o que fazer só lendo a tela, a interface está boa. Se ela travar, o problema não é o código, são as mensagens.